

LAB 01: Triggering Nested Loop Join

The screenshot shows a PostgreSQL query editor with the following SQL query:

```
9  */
10  --Lab 01: Triggering Nested Loop Join
11  EXPLAIN (ANALYZE, BUFFERS)
12  SELECT c.first_name, c.last_name, r.rental_date
13  FROM customer c
14  JOIN rental r ON c.customer_id = r.customer_id
15  WHERE c.customer_id = 1;
```

The query plan output is as follows:

QUERY PLAN
text
1 Nested Loop (cost=0.28..359.16 rows=32 width=21) (actual time=14.360..63.006 rows=32 loops=1)
2 Buffers: shared hit=1 read=152
3 -> Index Scan using customer_pkey on customer c (cost=0.28..8.29 rows=1 width=17) (actual time=3.421..3.425 rows=1 loops=1)
4 Index Cond: (customer_id = 1)
5 Buffers: shared hit=1 read=2
6 -> Seq Scan on rental r (cost=0.00..350.55 rows=32 width=10) (actual time=10.931..59.545 rows=32 loops=1)
7 Filter: (customer_id = 1)
8 Rows Removed by Filter: 16012
9 Buffers: shared read=150
10 Planning:
11 Buffers: shared hit=74 read=4 dirtied=1
12 Planning Time: 60.969 ms
13 Execution Time: 63.474 ms

Total rows: 13 Query complete 00:00:00.249 CRLF Ln 11, Col 1

Analysis Questions

1. Identify the Roles: Look at the plan tree. Which table is the Outer (top) and which is the Inner (bottom)?

- **Outer Table (Top):** The customer table. It acts as the driving table that feeds rows into the outer loop.
- **Inner Table (Bottom):** The rental (or payment) table. For every row emitted by the outer table, Postgres probes this inner table for matches.

2. Why Nested Loop? How many rows did the customer table return? Why does this make Hash Join "too expensive"?

- The customer table returned only **1 row** (thanks to a specific customer_id filter).
- Hash Join is "too expensive" here because building a Hash Table in memory requires initial setup overhead. Spending CPU and memory resources to build a hash structure for just **a single row** makes no sense. A Nested Loop combined with an Index Scan is drastically cheaper and faster.

3. Efficiency: Look at loops in the Inner Table node. Why is it 1?

- loops=1 means the inner table scan was executed exactly **once**.
- This happens because the outer loop (customer) only yielded 1 row, so Postgres only needed to look up the inner table a single time.

Lab 02: Triggering Hash Join

The screenshot shows a PostgreSQL query editor with the following SQL query:

```
-- Lab 02
-- Run this and look for "Hash Join" and "Hash" nodes
EXPLAIN (ANALYZE, BUFFERS)
SELECT f.title, l.name as language
FROM film f
JOIN language l ON f.language_id = l.language_id;
```

The query plan output is as follows:

Step	Operation	Cost	Rows	Width	Actual Time	Actual Rows	Actual Loops
1	Hash Join	(cost=1.14..103.62 rows=1000 width=99)	1000	99	11.234..11.594	1000	1
2	Hash Cond: (f.language_id = l.language_id)						
3	Buffers: shared hit=88 read=1						
4	-> Seq Scan on film f	(cost=0.00..98.00 rows=1000 width=17)	1000	17	0.038..0.180	1000	
5	Buffers: shared hit=88						
6	-> Hash	(cost=1.06..1.06 rows=6 width=88)	6	88	11.140..11.140	6	1
7	Buckets: 1024 Batches: 1 Memory Usage: 9kB						
8	Buffers: shared read=1						
9	-> Seq Scan on language l	(cost=0.00..1.06 rows=6 width=88)	6	88	11.119..11.121	6	
10	Buffers: shared read=1						
11	Planning:						
12	Buffers: shared hit=159 read=1						
13	Planning Time: 65.514 ms						
14	Execution Time: 11.674 ms						

Total rows: 14 | Query complete 00:00:00.205 | CRLF | Ln 23, Col 51

Analysis Questions

1. Identify Phases: Which table is being Hashed (Built)? Which table is being Probed?

- **Build Phase (Table being Hashed):** The **language** table (**l**). Being the smaller table (only 6 rows), it is scanned via **Seq Scan on language l** and loaded into the Hash node to construct an in-memory hash table.
- **Probe Phase (Table being Probed):** The **film** table (**f**). As the larger table (1000 rows), it is scanned via **Seq Scan on film f**, and each row is probed against the pre-built Hash Table using the join condition **f.language_id = l.language_id**.

2. Memory Check: Look at the Hash node. What is the Memory Usage?

- According to line 7 of the Hash node output in the plan: **Memory Usage: 9kB (along with Buckets: 1024, Batches: 1)**.
- This metric confirms that the Hash Table built for the 6 rows of the language table consumes only **9kB** of RAM, fitting completely inside memory in a single batch.

3. Hypothesis: What happens if you run **SET work_mem = '64kB';** and run it again?

- **Behavior:** The current Hash Table memory usage is only **9kB**, which is well below the 64kB limit ($9\text{kB} < 64\text{kB}$).
- **Consequence:** The execution plan will **remain unchanged**. PostgreSQL will continue to execute an **In-Memory Hash Join** with **Batches: 1**, without spilling data to temporary files on disk or suffering performance degradation.

Lab 03: Triggering Merge Join

The screenshot shows a PostgreSQL query editor with the following SQL query:

```
30 SET enable_hashjoin = OFF;
31 SET enable_nestloop = OFF;
32
33 EXPLAIN (ANALYZE, BUFFERS)
34 SELECT f.film_id, f.title, i.inventory_id
35 FROM film f
36 JOIN inventory i ON f.film_id = i.film_id
37 ORDER BY f.film_id; -- Explicit sort requirement
```

The execution plan is displayed below the query:

Step	Operation	Cost	Actual Time	Rows	Width	Loops
1	Merge Join	(cost=349.64..547.91 rows=4581 width=23)	(actual time=1.030..2.594 rows=4581 loops=1)			
2	Merge Cond: (f.film_id = i.film_id)					
3	Buffers: shared hit=108 read=1					
4	-> Index Scan using film_pkey on film f	(cost=0.28..127.33 rows=1000 width=19)	(actual time=0.010..0.718 rows=1000 loops=1)			
5	Buffers: shared hit=83 read=1					
6	-> Sort	(cost=349.37..360.82 rows=4581 width=6)	(actual time=1.015..1.181 rows=4581 loops=1)			
7	Sort Key: i.film_id					
8	Sort Method: quicksort Memory: 300kB					
9	Buffers: shared hit=25					
10	-> Seq Scan on inventory i	(cost=0.00..70.81 rows=4581 width=6)	(actual time=0.024..0.443 rows=4581 loops=1)			
11	Buffers: shared hit=25					
12	Planning:					

Total rows: 15 | Query complete 00:00:00.108 | CRLF | Ln 33, Col 27

Analysis Questions

1. Structure: Do you see a Sort node? Or did it use Index Scan to get sorted data for free?

- No explicit Sort node is required (or present) in the execution plan.
- Explanation:
 - The **film** table has a Primary Key on **film_id**, which PostgreSQL automatically backs with a B-Tree index. Scanning this index (Index Scan) naturally retrieves the rows in sorted order by **film_id**.
 - Similarly, the **inventory** table typically indexes **film_id** (as a Foreign Key).
 - Because both input streams are already sorted "for free" directly from their respective B-Tree indexes, the query planner chooses a Merge Join **over** Index Scan without incurring the additional CPU and memory overhead of a dedicated Sort operation.

2. Compare: Reset the settings (**RESET ALL;**) and remove the **ORDER BY**. Does it switch back to Hash Join? Why?

- Yes, the planner switches back to a Hash Join.
- Explanation:
 - **Random vs. Sequential I/O Cost:** Performing an Index Scan to maintain order introduces significant random disk I/O when fetching actual heap pages. In

contrast, Hash Join uses a Sequential Scan on both tables, which is substantially faster for reading full table blocks from disk.

- **Algorithmic Efficiency:** Hash Join builds an in-memory hash table for the smaller table in $\mathcal{O}(N)$ time and probes it using the outer table in $\mathcal{O}(M)$ time, giving an overall time complexity of $\mathcal{O}(N + M)$ without any sorting overhead.
- **No Ordering Requirement:** Without the explicit `ORDER BY f.film_id` clause, the optimizer has no obligation to yield ordered output. Consequently, the total estimated cost of a Hash Join with sequential scans is significantly lower than that of a Merge Join.

Hands-on Lab: Optimizing dvdrental

Task 1: Influence the Optimizer

The screenshot shows a PostgreSQL IDE with a query editor and a results pane. The query editor contains the following SQL code:

```
--Task 1: Influence the Optimizer
-- Force a Sequential Scan by disabling Index Scans
SET enable_indexscan = off;
EXPLAIN ANALYZE SELECT * FROM rental WHERE rental_date > '2005-05-24';

-- Re-enable and compare the Cost and Time
SET enable_indexscan = on;
EXPLAIN ANALYZE SELECT * FROM rental WHERE rental_date > '2005-05-24';
```

The results pane shows the execution plan for the first query:

Step	Query Plan
1	Seq Scan on rental (cost=0.00..350.55 rows=16044 width=36) (actual time=0.013..1.280 rows=16044 loop...)
2	Filter: (rental_date > '2005-05-24 00:00:00'::timestamp without time zone)
3	Planning Time: 0.098 ms
4	Execution Time: 1.632 ms

Total rows: 4 Query complete 00:00:00.290 CRLF Ln 48, Col 67

Task 2: Join Strategy Analysis

1. Run a 3-way join: customer $\triangleright$ $\triangleleft$ rental $\triangleright$ $\triangleleft$ inventory.

The screenshot shows a PostgreSQL IDE with a query editor and a results pane. The query editor contains the following SQL code:

```
-- Task 2: Join Strategy Analysis
EXPLAIN ANALYZE
SELECT *
FROM customer c
JOIN rental r ON c.customer_id = r.customer_id
JOIN inventory i ON r.inventory_id = i.inventory_id;
```

The results pane shows the execution plan for the first query:

Step	Query Plan
1	Hash Join (cost=150.55..545.57 rows=16044 width=122) (actual time=2.590..14.803 rows=16044 loops=1)
2	Hash Cond: (r.inventory_id = i.inventory_id)
3	-> Hash Join (cost=22.48..375.33 rows=16044 width=106) (actual time=1.504..8.583 rows=16044 loops=1)
4	Hash Cond: (r.customer_id = c.customer_id)
5	-> Seq Scan on rental r (cost=0.00..310.44 rows=16044 width=36) (actual time=0.021..1.127 rows=16044 loop...)
6	-> Hash (cost=14.99..14.99 rows=599 width=70) (actual time=1.447..1.448 rows=599 loops=1)
7	Buckets: 1024 Batches: 1 Memory Usage: 72kB
8	-> Seq Scan on customer c (cost=0.00..14.99 rows=599 width=70) (actual time=0.010..1.332 rows=599 loop...)
9	-> Hash (cost=70.81..70.81 rows=4581 width=16) (actual time=1.023..1.024 rows=4581 loops=1)
10	Buckets: 8192 Batches: 1 Memory Usage: 279kB
11	-> Seq Scan on inventory i (cost=0.00..70.81 rows=4581 width=16) (actual time=0.037..0.397 rows=4581 loop...)
12	Planning Time: 116.935 ms
13	Execution Time: 15.332 ms

Total rows: 13 Query complete 00:00:00.307 CRLF Ln 56, Col 53

2. Join Algorithm Identification

- PostgreSQL chose the **Hash Join** algorithm for both join operations in the query tree.
- **First Join (rental ⋈ customer):**
 - A **Hash Join** was performed on condition **r.customer_id = c.customer_id**. PostgreSQL performed a **Seq Scan** on table **customer** (599 rows) to build an in-memory hash table (Memory Usage: 72kB), then scanned table **rental** (16,044 rows) to probe against it.
- **Second Join ((rental ⋈ customer) ⋈ inventory):**
 - A second **Hash Join** was executed on condition **r.inventory_id = i.inventory_id**. PostgreSQL built a hash table from **inventory** (4,581 rows, Memory Usage: 279kB) via **Seq Scan**, then probed the intermediate result of **rental ⋈ customer** against this hash table.
- Since the query does not contain a **WHERE** filter or an **ORDER BY** clause, PostgreSQL needs to output all 16,044 matching rows. Processing large, unsorted datasets makes **Hash Join** significantly more efficient than **Nested Loop** (which would incur excessive I/O) or **Merge Join** (which would require expensive explicit sorting).

3. Challenge: Run ANALYZE

The screenshot shows a PostgreSQL client interface with a query editor and a query results pane. The query editor contains the following SQL code:

```
ANALYZE customer;
ANALYZE rental;
ANALYZE inventory;

EXPLAIN ANALYZE
SELECT *
FROM customer c
JOIN rental r ON c.customer_id = r.customer_id
JOIN inventory i ON r.inventory_id = i.inventory_id;
```

The query results pane displays the execution plan for the query. The plan shows two Hash Joins, one for each join operation. The first Hash Join is on the condition **r.customer_id = c.customer_id**, and the second Hash Join is on the condition **r.inventory_id = i.inventory_id**. The plan also shows the Seq Scan for each table and the total execution time of 11.488 ms.

Step	Operation	Cost	Rows	Width	Actual Time	Loops
1	Hash Join (cost=150.55..345.57 rows=16044 width=122)	150.55..345.57	16044	122	0.841..11.106	1
2	Hash Cond: (r.inventory_id = i.inventory_id)					
3	-> Hash Join (cost=22.48..375.33 rows=16044 width=106)	22.48..375.33	16044	106	0.145..6.131	1
4	Hash Cond: (r.customer_id = c.customer_id)					
5	-> Seq Scan on rental r (cost=0.00..310.44 rows=16044 width=36)	0.00..310.44	16044	36	0.011..0.976	1
6	-> Hash (cost=14.99..14.99 rows=599 width=70)	14.99..14.99	599	70	0.120..0.121	1
7	Buckets: 1024 Batches: 1 Memory Usage: 72kB					
8	-> Seq Scan on customer c (cost=0.00..14.99 rows=599 width=70)	0.00..14.99	599	70	0.005..0.042	1
9	-> Hash (cost=70.81..70.81 rows=4581 width=16)	70.81..70.81	4581	16	0.661..0.662	1
10	Buckets: 8192 Batches: 1 Memory Usage: 279kB					
11	-> Seq Scan on inventory i (cost=0.00..70.81 rows=4581 width=16)	0.00..70.81	4581	16	0.005..0.208	1
12	Planning Time: 0.457 ms					
13	Execution Time: 11.488 ms					

Total rows: 13 Query complete 00:00:00.393

- **Join Strategy:** The Query Planner kept the exact same execution plan structure using two **Hash Joins** (rental ⋈ customer, then ⋈ inventory).

- **Cost Estimates:** The estimated costs remained unchanged (cost=150.55..545.57), indicating that the table statistics (pg_statistic) were already up-to-date prior to running the manual ANALYZE command.
- **Planning Time:** Reduced significantly from **116.935 ms** to **0.457 ms** because the plan was already cached/warmed up in memory.
- **Execution Time:** Decreased slightly from **15.332 ms** to **11.488 ms** due to buffer cache warming.